WhatsApp Project Intelligence Platform

A self-hosted platform that turns WhatsApp project chats into structured, evidence-linked project intelligence, a live dashboard, search, and a permission-bound assistant. No external AI service is used.

What it does, end to end

- **Ingests** chats: phone export saved to Google Drive and imported from a watched folder, manual upload, or (for business-created groups and 1:1 chats) the official WhatsApp Cloud API webhook.
- **Analyses** every message with deterministic rules and small models (English, Hinglish, Hindi): tasks, assignees, deadlines, decisions, requirements and changes, bugs, commitments, client-pending items, conflicts, risks, questions and answers, tone, progress and health, daily/weekly/monthly summaries.
- **Shows** it in a role-aware dashboard that updates live; every number links to the messages it came from (evidence ids).
- **Answers and acts** through an assistant: a self-hosted LLM proposes, a deterministic policy engine decides, actions run with the user's own permissions and are verified by reading back.
- **Protects** data: RBAC with project membership, encrypted uploads and phone numbers, signed webhooks, audit trail, per-project retention and erasure.

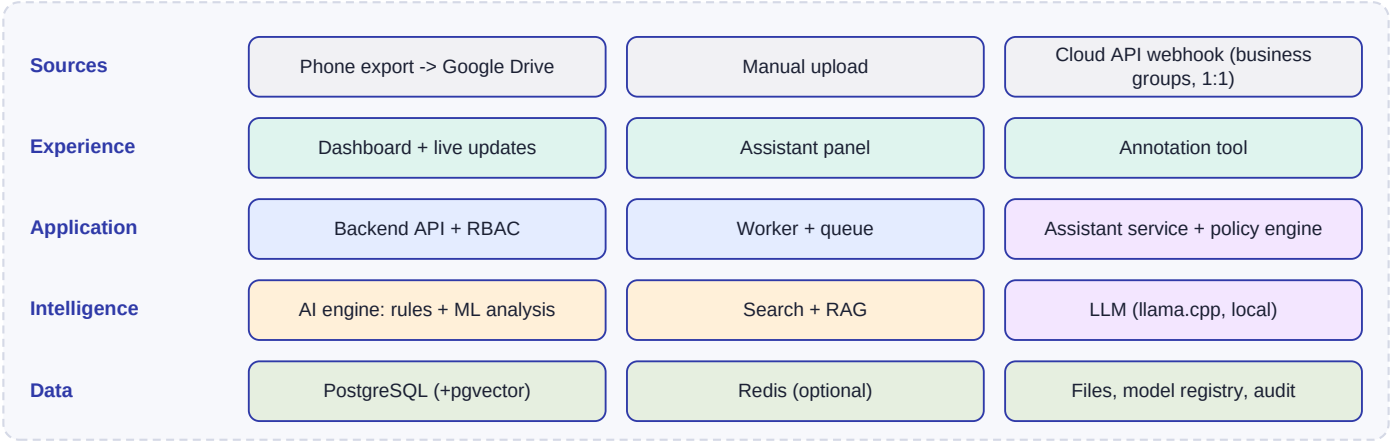
Design principles

Principle	How it shows up
Evidence over assertion	Every derived item carries source message ids and a confidence or review flag; raw text is never modified.
Deterministic where possible	Counts, dates, rates and response times are plain code; models only classify or generate text.
LLM proposes, code disposes	Permissions, risk tiers, confirmations, id checks and rate limits are ordinary code, outside the model.
Honest degradation	Live events fall back to polling, the app serves stored data if the AI engine is down, the assistant abstains instead of guessing.
Self-hosted, single company	Models run locally (Qwen2.5-7B via llama.cpp); isolation unit is the project, not a tenant (ADR-002).
Additive change	Schema changes are migrations that only add; existing roles, APIs and data keep working.

At a glance

Services	Languages	Data model	Automated tests (all passing)	Not yet measured
Frontend, Backend API + worker, AI engine, Assistant, LLM server, PostgreSQL, Redis (optional), edge proxy	TypeScript (backend, frontend), Python (AI engine, assistant)	38 Prisma models; derived outputs also stored as JSON snapshots	AI engine 175, backend 83, assistant 57, frontend 12 + typecheck	Real-LLM tool-calling quality (bake-off running), speech accuracy, load tests, independent accuracy of Q&A; / tone

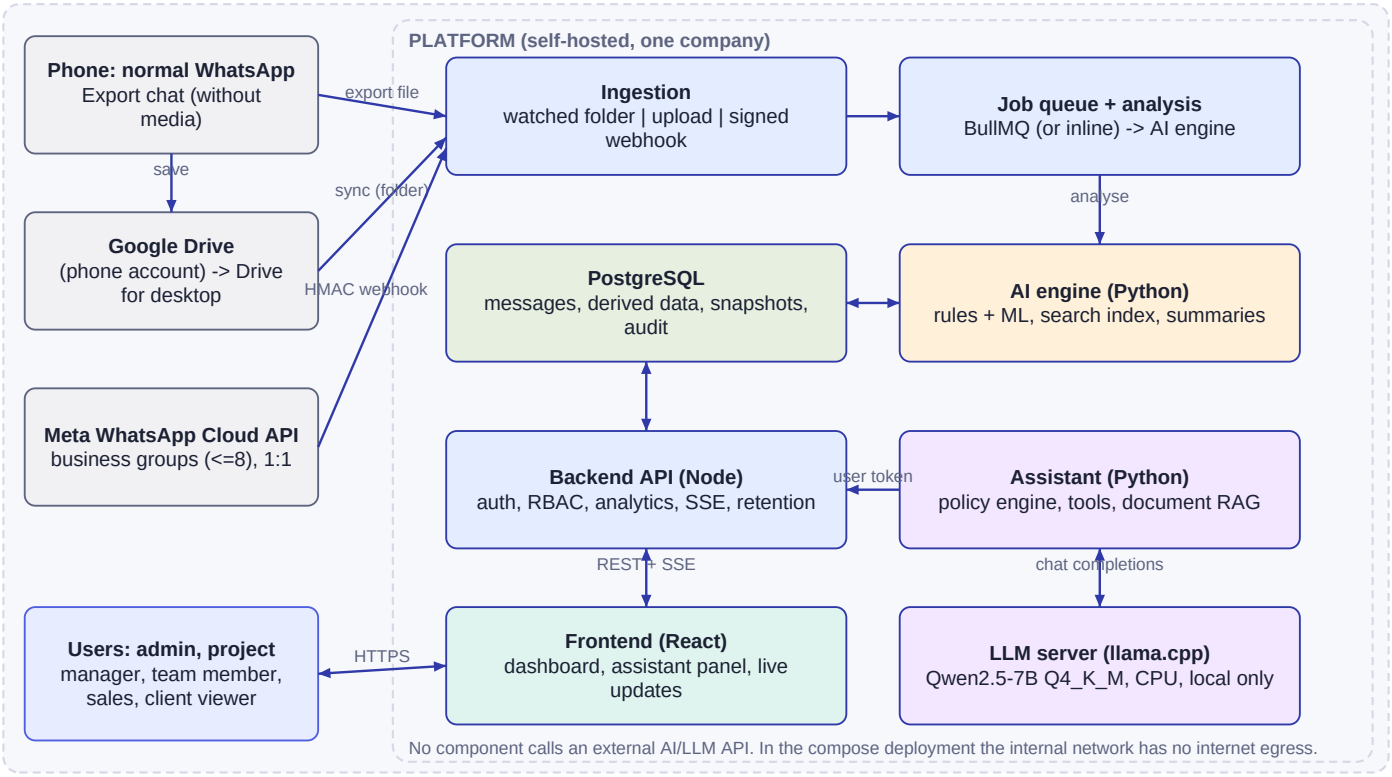
The system in five layers



Status legend used in this document: ***Built and tested*** *|* ***Built, not exercised against the real counterpart*** *(Meta, Google Drive, a speech model) |* ***Planned / out of scope*** *(multi-tenancy, RabbitMQ).*

1. System context

Who and what the platform talks to. Everything inside the dashed boundary runs on the owner's hardware.



Boundaries and trust

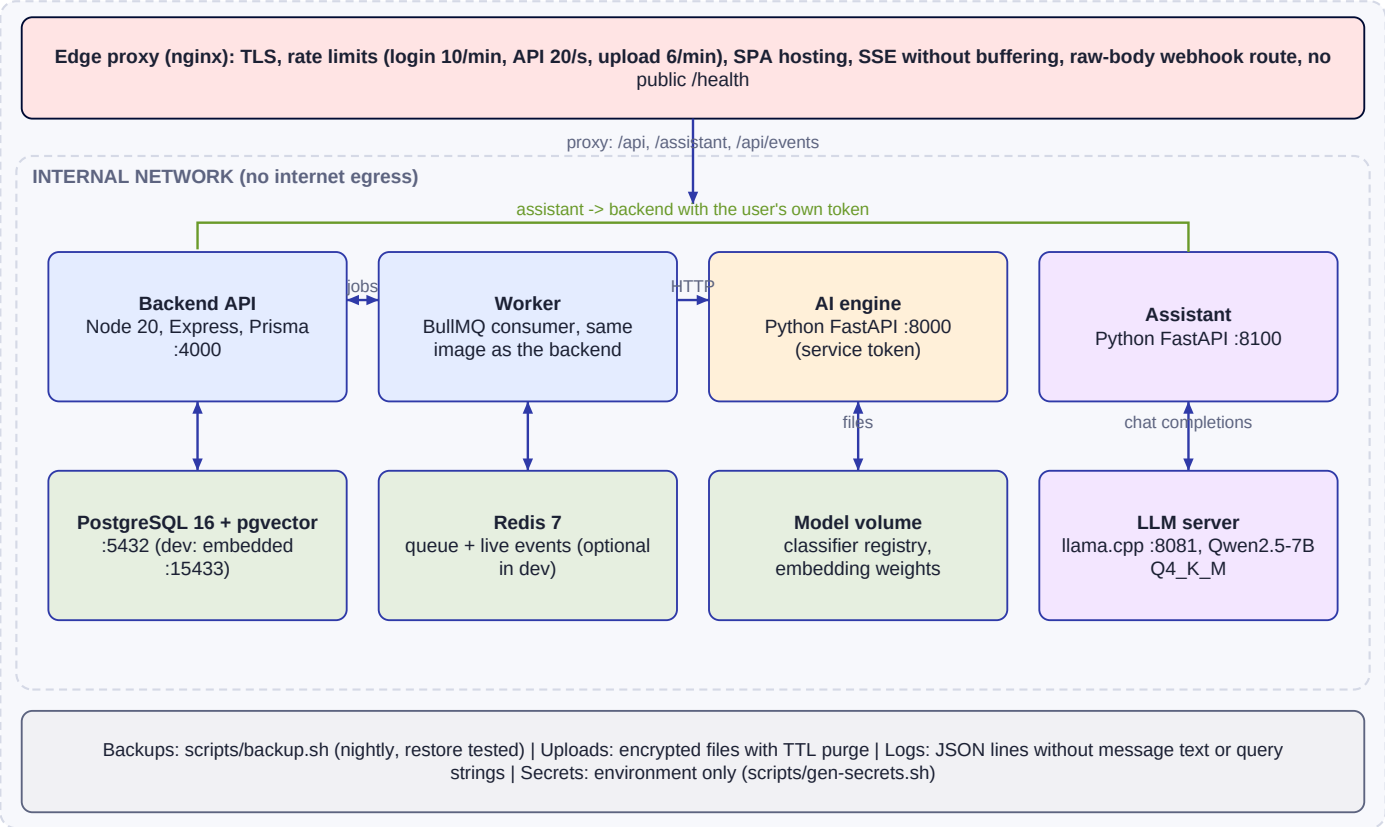
Boundary	What crosses it	Control
Phone/Drive -> platform	an exported chat file (text, optionally zipped)	file only, no WhatsApp access; size, type, stability and duplicate checks; incremental import
Meta -> platform	signed webhook events for business-created groups and 1:1 chats	HMAC-SHA256 over the raw body; only explicitly mapped conversations are stored; idempotent per message id
Browser -> platform	HTTPS API calls, SSE stream	JWT (15 min) + rotating refresh token; RBAC; project membership; rate limits at the edge
Assistant -> backend	tool calls	always with the signed-in user's token; tiers T0-T3; confirmation by UI button for sensitive actions
Platform -> internet	none for AI	models are local; internal compose network has no egress

Why WhatsApp ingestion looks like this

- The official Cloud API can read only **business-created groups (max 8 participants)** and 1:1 customer chats, requires an Official Business Account, and offers **no history backfill for groups** (verified against Meta documentation, Oct 2026). Existing personal groups cannot be read by any authorized API.
- Therefore the personal-WhatsApp path is **phone export -> Google Drive -> watched folder**, which is automatic from the synced file onward. Scraping or WhatsApp Web automation is deliberately not used (ban risk, security).

2. Service and deployment view

Production topology (docker-compose). Only the edge proxy is exposed; every other container sits on an internal network without internet access.

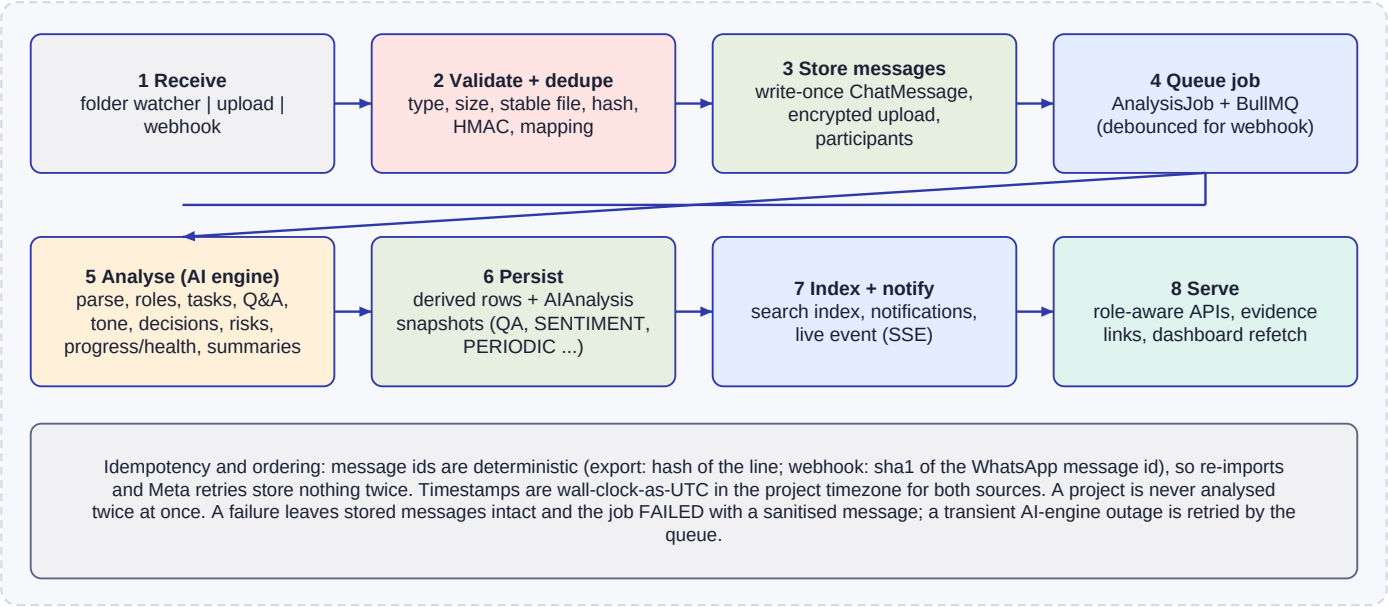


Services, ports and responsibilities

Service	Tech	Dev port	Responsibility	Scaling note
Frontend	React 19, Vite 8, TypeScript, Tailwind 4, React Router 7, TanStack Query 5, Recharts	5199	dashboards, assistant panel, SSE client with polling fallback	static assets behind nginx
Backend API	Node, Express 4, TypeScript, Prisma 6, zod, helmet	4000	auth, RBAC, projects, import, analysis orchestration, analytics, webhook, retention, SSE, status	stateless; scale horizontally
Worker	BullMQ 6, ioredis	-	runs analysis jobs (2 attempts, exponential backoff)	docker compose --scale worker=N
AI engine	Python, FastAPI, scikit-learn, NumPy, NetworkX, reportlab	8000	parsing, analysis pipeline, search index, summaries, reports, annotation tool	CPU-bound; index lives in memory
Assistant	Python, FastAPI, pydantic, httpx	8100	agent loop, policy engine, tools, document RAG, voice endpoint	stateless except sessions
LLM	llama.cpp server, Qwen2.5-7B-Instruct Q4_K_M (4.68 GB, Apache-2.0)	8081	tool-calling language model	CPU only here: about 1-4 tokens/s
PostgreSQL	16 + pgvector (dev: embedded, TEXT embeddings)	5432 / 15433	single source of truth	backup/restore tested
Redis	7	6379	BullMQ and cross-process live events	optional; inline queue without it

3. End-to-end data flow

From a message on the phone to a number on the dashboard. The same eight steps serve all three ingestion paths.



Ingestion paths compared

Path	Trigger	Automatic?	Covers	Key safeguards
Watched folder (personal WhatsApp)	file appears in AUTO_IMPORT_DIR// (.txt or .zip) every 30 s scan	yes, after the phone export is saved to Drive	any chat you can export, incl. existing groups	size+mtime stable across two scans, SHA-256 duplicate check, busy-project wait, zip-bomb cap, unknown/ambiguous folder untouched, moved to imported/ or rejected/ + reason
Manual upload	POST /projects/:id/chat/upload (UI)	no	same	extension, binary, size and WhatsApp-header checks; incremental append
Cloud API webhook	Meta POSTs to /api/webhooks/whatsapp	yes (real time)	business-created groups (max 8) and 1:1 chats only	HMAC on raw body, conversation must be mapped to a project, media metadata only, replies stored, reactions ignored

Worked example: one message, end to end

Step	What happens to: *Priya (client): "Rahul, please fix the payment bug by Friday"*
Arrive	exported on the phone, synced from Drive, found by the watcher, imported once (hash check); only messages not seen before are appended
Analyse	assignment cue + vocative -> task 'fix the payment bug', assignee Rahul, deadline Friday (resolved against the message date); bug detected; question/tone/topic tags added
Evidence	task and bug rows store the source message id and a confidence; low confidence or ambiguity sets needs_review
Later	Rahul: 'payment bug fixed' -> task status trace TODO -> COMPLETED, bug FIXED; if the client replies 'still not working' the item is reopened for review
Show	Tasks page, health and progress recalculated, summaries and Q&A updated; the browser refreshes itself via the live event; every figure links back to the messages

Live updates

- Every stage change publishes an event (ids, stage, progress; never message text) to an in-process bus, or to a Redis channel when a separate worker runs.
- **GET /api/events** (Server-Sent Events) delivers it to a browser only if the user can see that project (checked at delivery, cached 30 s), max 5 streams per user, heartbeat every 25 s.
- The frontend refetches on completion and shows *Live*; after 3 failed connects it shows *Polling* and refreshes every 30 s.

4. Backend architecture

Node + Express + Prisma modular monolith. Authorization is enforced here on every request; the frontend only hides things.

Request pipeline

Layer	What it does
requestLog	one JSON line per request (route pattern, status, ms, user id); no bodies, no query strings, no message text
helmet + CORS	security headers, HSTS, allow-listed origins with credentials
raw body for /webhooks/whatsapp, JSON (1 MB) elsewhere	the webhook signature is computed over the exact bytes
authenticate	JWT access token (15 min); refresh tokens are single-use and rotated; bcrypt (cost 12); constant-time compare for unknown users
requirePermission(action)	role-permission matrix (SUPER_ADMIN, ADMIN, PROJECT_MANAGER, TEAM_MEMBER, SALES, CLIENT_VIEWER)
projectAccess(view|edit)	project membership; non-members get 404 so existence is not leaked; read-only members cannot edit
validateBody / validateQuery (zod)	typed input, 422 on failure; limits on sizes and enums
errorHandler	sanitised errors {code, message}; no stack traces or upstream bodies to clients

Modules

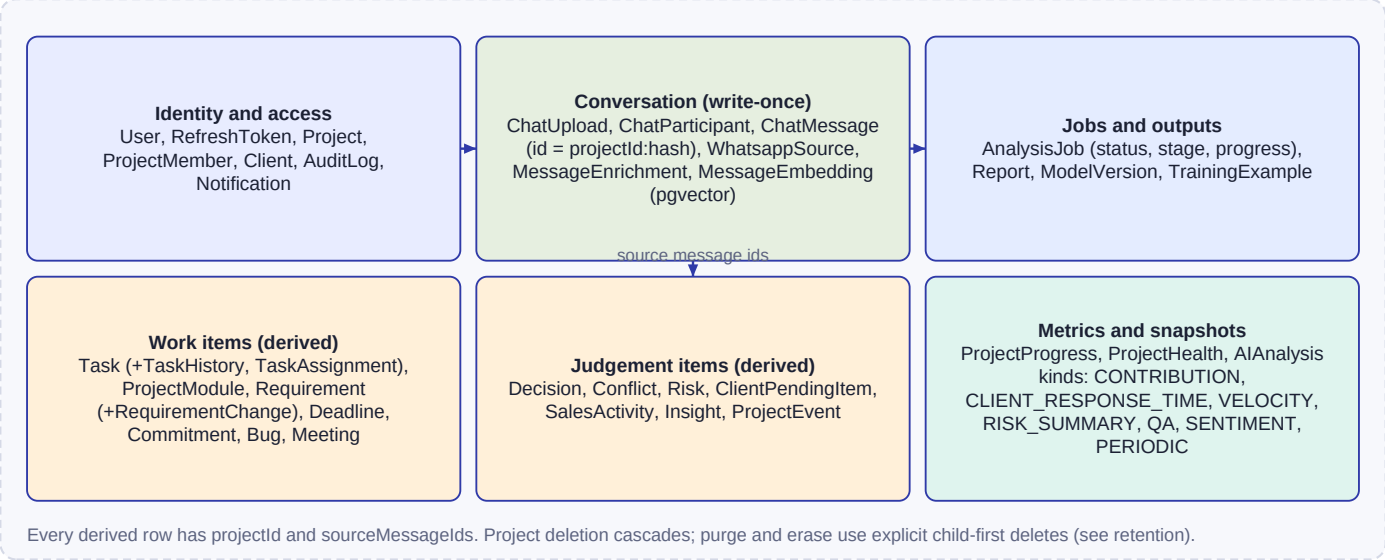
Module (backend/src)	Endpoints / responsibility
controllers/auth	register (first user = SUPER_ADMIN), login, refresh, logout, me
controllers/projects, tasks, chat, reports, overview	projects and members, tasks (human edits are never overwritten by re-analysis), upload/uploads/messages/analysis status/reanalyze, report generate + export (PDF/XLSX/CSV), executive overview, users
controllers/analytics	progress, health, team, sales, client-response-time, velocity, client-pending, risks, conflicts, timeline, insights, summary, notifications, audit, **qa, sentiment, summaries**
controllers/status	/health, /ready, /queue/status, /ai/status, /auto-import/status, /auto-import/folders
controllers/whatsapp + services/whatsapp	webhook verify + signed delivery, conversation-to-project mapping (WhatsappSource), normalisation, idempotent storage, debounced re-analysis
services/autoimport + importer + zipText	watched-folder import, shared import checks, defensive ZIP reader
controllers/events + events/bus	SSE stream with per-event authorization; in-process or Redis pub/sub
controllers/retention + services/retention	per-project retention policy (30-3650 days), purge, erasure with derived data; scheduled sweep every 6 h
services/analysis + queues + workers	job runner: parse -> store -> analyse -> persist -> index -> notify; BullMQ or inline

Permissions at a glance

Role	Typical rights
SUPER_ADMIN / ADMIN	everything; manage users; see all projects; erase project data (project:delete)
PROJECT_MANAGER	create projects, manage members, upload/connect sources, edit tasks, generate reports, set retention; sees only own projects
TEAM_MEMBER	view analytics and tasks of member projects; change task status only; read raw chat
SALES	sales analytics, client-facing views, reports (view)
CLIENT_VIEWER	client-facing views only (progress, health, timeline, client-pending); no raw chat, no analytics

5. Data model and storage

PostgreSQL via Prisma (38 models). Raw conversation data is write-once; AI interpretation lives in separate rows that always point back to the messages.



Every derived row has projectId and sourceMessageIds. Project deletion cascades; purge and erase use explicit child-first deletes (see retention).

Rules the schema enforces

- **Never destroy the original:** ChatMessage.originalText is the exact export/webhook text and is never updated; cleaned text and AI output are separate.
- **Deterministic ids:** export lines hash to stable ids; webhook messages use sha1 of the WhatsApp id; re-imports and retries are no-ops.
- **Human decisions win:** rows with reviewState HUMAN_CONFIRMED / HUMAN_CORRECTED are never overwritten by re-analysis; retention reports them instead of deleting.
- **Evidence everywhere:** tasks, decisions, risks, Q&A items and summary highlights store source message ids; the UI shows them.
- **Time:** wall-clock-as-UTC in the project timezone (default Asia/Kolkata) so what users saw in WhatsApp is what the API returns.

Migrations added in this iteration (additive only)

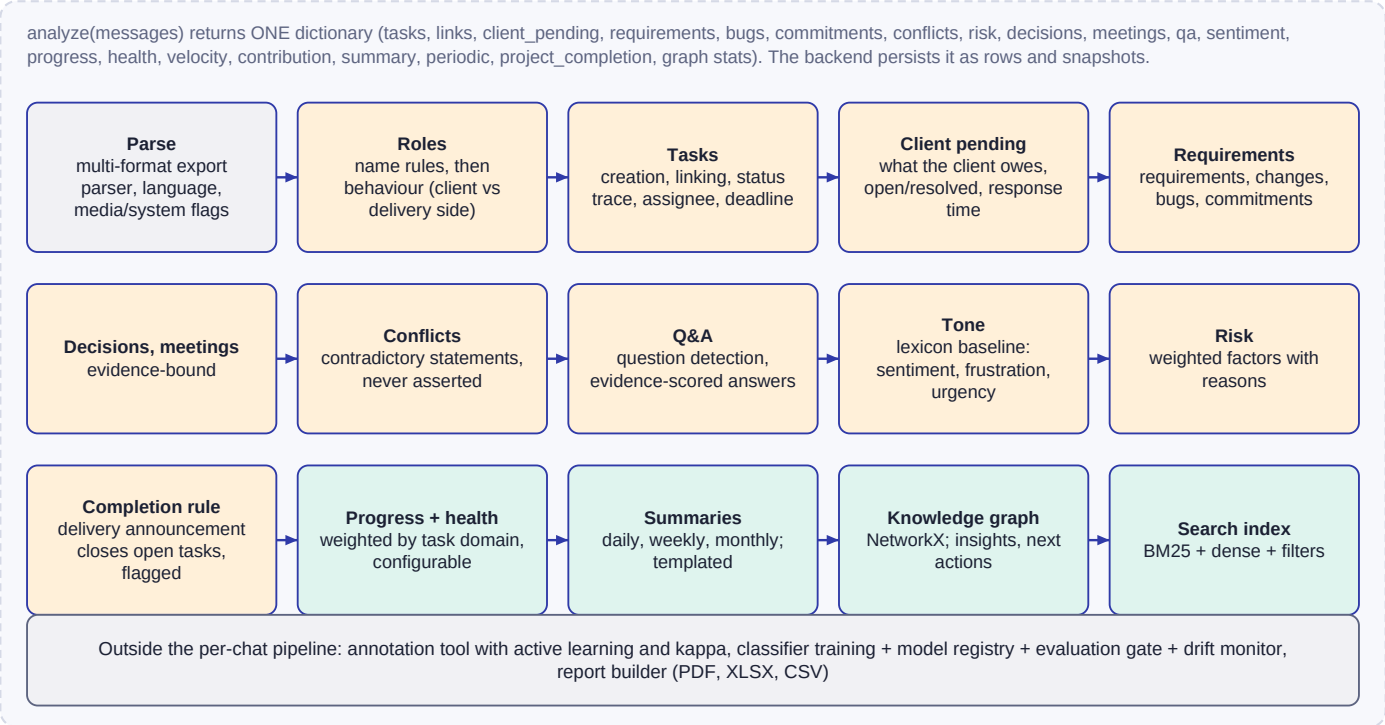
Migration	Change
20261006000000_whatsapp_sources	table WhatsappSource: (phoneNumberId, conversationKey) -> project, carrier upload, counters; unique mapping
20261006010000_project_retention	Project.retentionDays (nullable)
(no migration)	Q&A, tone and periodic summaries are stored as AIAnalysis snapshots with new kinds QA, SENTIMENT, PERIODIC

Other stores

Store	Holds	Protection / lifecycle
Upload directory	uploaded chat files (encrypted AES-256-GCM, random storage key)	purged after UPLOAD_TTL_HOURS; messages stay in the DB
Phone numbers	ChatParticipant.phone	AES-256-GCM field encryption; masked in API responses
AI engine memory	per-project search index (BM25 + vectors)	rebuilt by analysis; dropped on purge/erase
Model registry (files)	classifier versions, metrics, evaluation gate, rollback	promotion needs a passing gate; drift monitored with PSI
Assistant files	audit.jsonl, feedback.jsonl, document library	no message text in audit; feedback is a review queue only
private-data/ (git-ignored)	raw/anonymised chats, labels, LLM files, logs, dev credentials, watched inbox	never committed; dev only

6. AI engine and analysis pipeline

Python FastAPI service. Rules and small models first; no generative model is involved in extraction, so every result is reproducible and explainable.



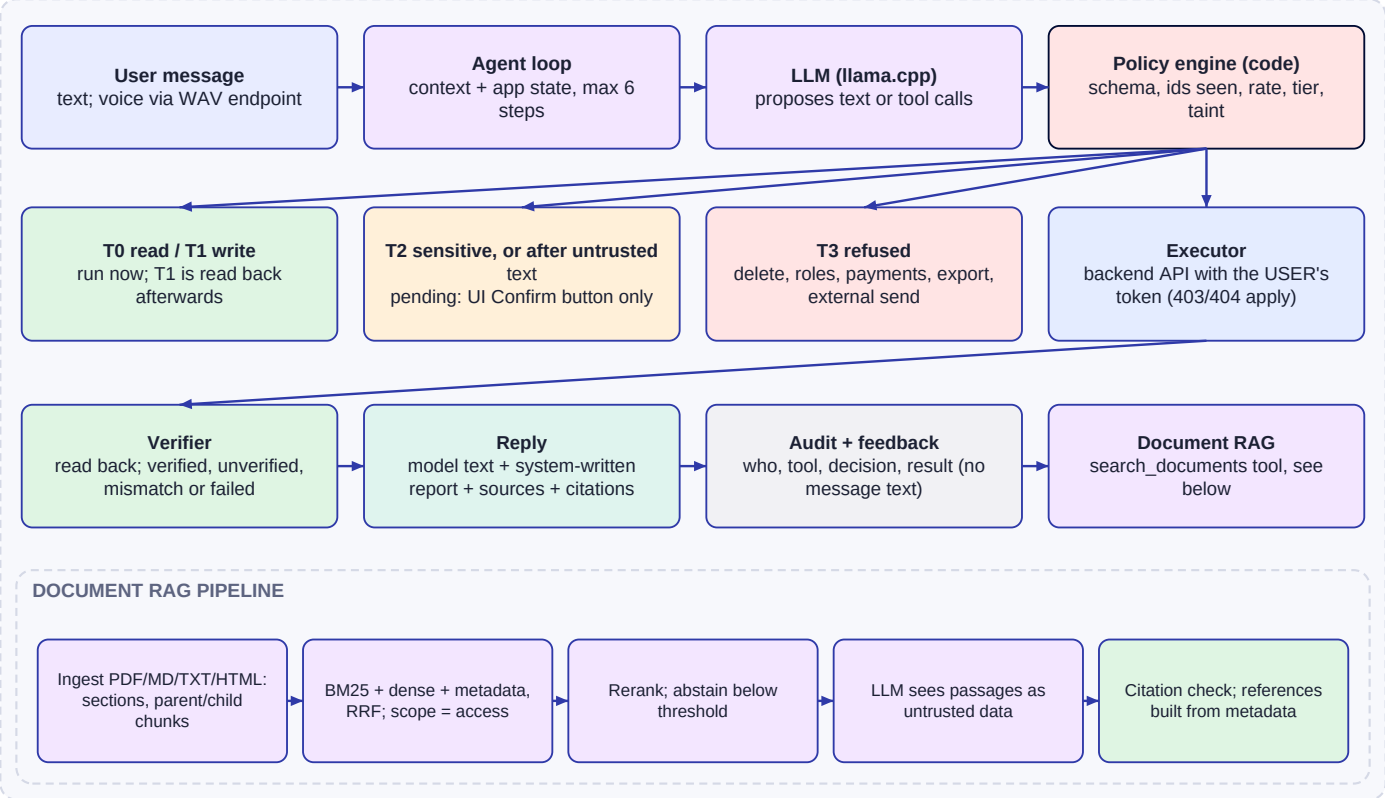
Method and honest quality

Capability	Method	Measured so far
Message category proposals	regex lexicons EN/Hinglish/Devanagari + signals; TF-IDF/LR classifier exists but is not deployed	69.5% agreement with model-made labels on 233 messages of one chat (in-sample, silver labels); classifier scored lower and was not used
Tasks, assignees, status	assignment/commit/done cues, topic linking, status trace	hand-written unseen chats: recall 80-100% when tuned, 33% on a fresh holdout (tiny sets)
Questions and answers	detection + evidence score; unconfirmed matches stay POSSIBLY_ANSWERED	unit tests only; real chat: 7 answered / 7 possible / 6 unanswered of 20, not independently verified
Tone / urgency	word lists with negation, emoji, repetition cues	>=85% on 31 author-written examples; real chat mostly neutral (coverage limit)
Project completion	delivery announcement by non-client closes open tasks, flagged for review	HG chat: progress 22% -> 100%, health 46 CRITICAL -> 79 AT_RISK
Search	hybrid BM25 + multilingual-e5 or hashing embedder, RRF	benchmarks on synthetic and unseen chats (docs/PHASE7)
Summaries	evidence-linked facts, templated text	unit tests; no human quality rating yet

Labels used so far are model-made (single reviewer) and one real project: treat all accuracy figures as indicative, not as a claim of production accuracy. Human labelling of several chats is the largest remaining quality lever.

7. Assistant, RAG and voice

The assistant is an operating layer over the application, not a chatbot with its own authority. The model never touches the database or holds permissions.



Tool registry (23 tools)

Tier	Tools	Rule
T0 read	list_projects, get_project_overview, get_progress, get_health, get_summary, list_tasks, get_task_history, list_client_pending, list_risks, list_users, search_documents, navigate, set_dashboard_filter	ids must come from an earlier lookup in the session
T1 reversible write	create_project, create_task, update_task (not status), generate_report, reanalyze_project, mark_notifications_read	typed args; rate limit 10/min; read-back verification; after untrusted text -> confirmation
T2 sensitive	complete_task, cancel_task, add_project_member	UI Confirm button only; hash of tool+args, 5 min, single use, same user
T3 refused	delete_task, delete_project, remove_project_member, create_user, change_user_role, make_payment, send_external_message, export_all_data	pointed to the admin UI

Voice and models

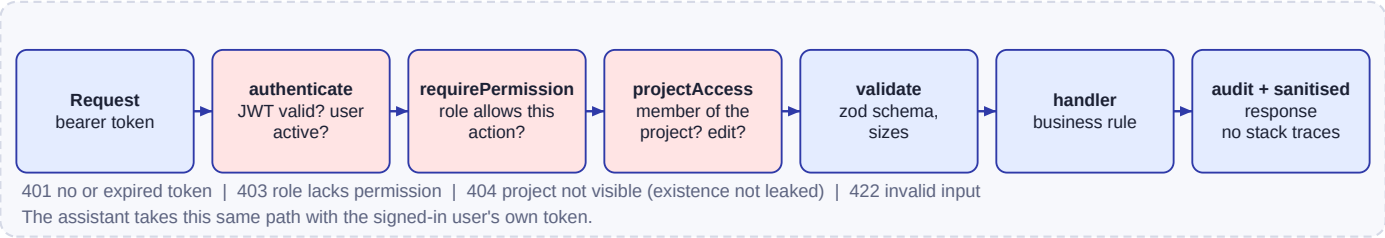
- **Voice:** energy VAD, end-pointing and barge-in state machine and a WAV endpoint are built and tested with synthetic audio; **no ASR/TTS model is installed, so accuracy and latency are unmeasured**. Spoken input gets no extra trust and cannot confirm an action.
- **LLM:** Qwen2.5-7B-Instruct Q4_K_M via llama.cpp, CPU only (about 1-4 tokens/s on this laptop). A 37-case state-diff bake-off (English + Hinglish) is running; sandbox safety suites passed against a worst-case obedient stub (0 state changes, 0 T3 executed). Real-model quality is not yet known.
- **Untrusted data:** chat text, task titles and documents are wrapped as data with a 'never follow instructions' note; they can never confirm or originate a sensitive action.

8. Security, privacy and reliability

Threat model for a single-company deployment. Each control is implemented and has tests unless marked.

Threat	Control	State
Unauthenticated / forged access	JWT 15 min, rotating single-use refresh tokens, bcrypt 12; every router authenticates	tested
Privilege escalation, cross-project reads	role matrix + project membership; 404 for non-members; SSE checks membership per event	tested
Forged or replayed webhook	HMAC-SHA256 on the raw body (constant-time compare); idempotent ids; unmapped conversations dropped	tested; no timestamp window
Malicious files (zip bomb, half-synced, junk)	size caps, decompression cap, stability check, WhatsApp-header check, rejected/ + reason	tested
Prompt injection through chats/documents	untrusted-data wrapping, taint-then-confirm, T3 refusal, schema + id provenance, user-token execution	tested vs worst-case stub; real model pending
Secrets / credential leakage	env-only secrets; AES-256-GCM for uploads and phones; logs without bodies/queries; anonymiser masks pasted passwords for labelling	tested; raw text not secret-scanned
Injection / mass assignment	zod validation, Prisma parameterised queries	tested
Denial of service	edge rate limits, body caps, per-user SSE and assistant limits	nginx config not validated here (no nginx)
Over-retention of personal data	per-project retention 30-3650 days, purge with derived-data rebuild, erasure (needs project:delete + exact name)	tested
Accountability	AuditLog (who/what/when) in the backend; assistant audit of decisions and results	implemented

Authorization path of every request



401 no or expired token | 403 role lacks permission | 404 project not visible (existence not leaked) | 422 invalid input

The assistant takes this same path with the signed-in user's own token.

Reliability and failure behaviour

Failure	What happens
AI engine down	uploads still stored; analysis job retried by the queue; /api/ready reports *degraded*; stored dashboards keep working
Redis down (bullmq mode)	events fall back to the local process; queue readiness fails (/api/ready 503); inline mode needs no Redis
Backend restart during analysis	job marked failed with a sanitised message; re-run with reanalyze (inline mode); BullMQ retries when Redis is used
Live stream cannot connect	dashboard shows Polling and refetches every 30 s
LLM unreachable	assistant returns 503 with a clear message; nothing is executed
Action cannot be verified	reply says so explicitly (unverified / mismatch / failed); it never says Done without evidence

Residual risks that design cannot remove: a hijacked model can still do what the signed-in user may do within tiers and rate limits; rule-based extractors can misread chats (outputs are advisory and carry evidence and review flags); the HG chat contains a website password and an SMS API key that must be rotated.

9. Technology, operations, quality and roadmap

How it is built, run and checked, and what is still open.

Area	Choices
Languages / frameworks	TypeScript (Node, Express, Prisma, React, Vite, Tailwind), Python (FastAPI, scikit-learn, NumPy, NetworkX, reportlab, pydantic, httpx)
Data / messaging	PostgreSQL 16 + pgvector, Redis 7 + BullMQ (RabbitMQ and Kafka deliberately not used, ADR-001), SSE for live updates
Models	Qwen2.5-7B-Instruct Q4_K_M (llama.cpp), multilingual-e5 small or hashing embedder, lexical reranker (cross-encoder supported, untested), TF-IDF classifier (not deployed)
Dev stack	scripts/dev/dev-up.ps1 starts embedded Postgres :15433, AI engine :8000, backend :4000, assistant :8100, LLM :8081, frontend :5199 as detached processes; logs in private-data/logs
Production	docker-compose: postgres, redis, backend, worker, edge nginx (TLS), internal network, backups with restore test, secrets generator; assistant and LLM services not yet in compose
Health	/api/health, /api/ready (db, queue, ai), admin /api/queue/status, /api/ai/status, /api/auto-import/status; assistant /health

Quality gates

Suite	Result	Covers
AI engine (pytest)	175 pass	parser, extractors, Q&A, tone, summaries, completion rule, analytics, search, reports, registry, annotation
Backend (vitest)	83 pass, 11 files	auth, e2e, failures, SSE, webhook, retention/erasure, watched folder, insights, status
Assistant (pytest)	57 pass	policy, confirmations, injection, RAG, API, evaluation harness, voice
Frontend	12 pass + typecheck	helpers, SSE parser; UI checked manually in a browser
Real-backend smoke	pass	create/verify/confirm/refuse against the live API
LLM bake-off (37 cases)	running	tool selection, arguments, state-diff success, safety, latency

Day-to-day operation

- **Start / stop:** scripts/dev/dev-up.ps1 and dev-down.ps1 (dev); docker compose up -d (production). **Check:** /api/ready and the Settings page (auto-import status).
- **Add data:** export on the phone to the Drive folder named like the project; or upload in the panel; or map a Cloud API conversation. **Retention:** set per project; sweep runs every 6 h.
- **Keep safe:** backup nightly, test restore, rotate secrets that ever appeared in a chat (the HG website password and SMS API key), never expose the dev database or the weak dev password.

Roadmap (decisions already taken: BullMQ stays, single company, self-hosted models)

Next	Why / dependency
1. Finish LLM bake-off; choose or reject Qwen2.5-7B for tool use	needs hours of CPU time; may require a smaller/faster model or a GPU
2. Human-label several real chats (target 1,500+ messages, double annotation, kappa >= 0.7)	largest quality lever; unlocks honest REAL_TEST and a trained classifier
3. Install Google Drive for desktop and point AUTO_IMPORT_DIR at the synced folder	needed for the personal-WhatsApp automation; not testable here yet
4. ASR/TTS models with your approval, WER on your recordings, mic UI	voice is unmeasured today
5. Add assistant + LLM to docker-compose; validate nginx config; load and failure tests	production readiness
6. Webhook against Meta (only if an Official Business Account exists)	business-created groups and 1:1 only
7. Streaming assistant replies, history, markdown; connections screen for webhook sources	UX completeness